

ML Basics & Optimization

발표자 : 문상혁

최종 업데이트 : 2026.07.01.

| Overview

[1] Model Class

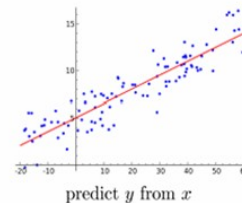
[2] Loss Function

[3] Optimizer

Types of Learning Problem

Supervised Learning

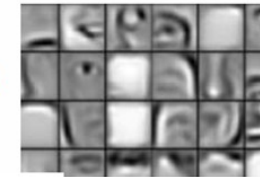
정답이 붙어있는 데이터로 학습하는 방식
Ex. Regression, SVM ...



$$\text{dog image} \xrightarrow{f_{\theta}(x) = y} \text{[object label]}$$

Unsupervised Learning

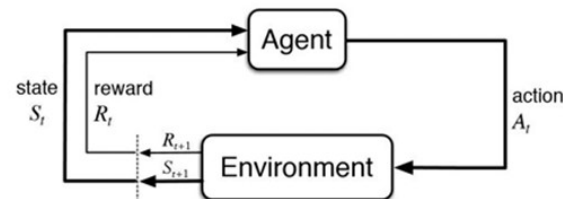
정답 레이블 없이 데이터의 구조를 학습하는 방식
Ex. Clustering, Generative Modeling ...



representation

Reinforcement Learning

행동(action)을 선택하고 보상(reward)를
최대화하는 학습 방식



Machine Learning method

[1] Define Model Class

어떻게 Model을 표현할지 결정한다.

[2] Define Loss Function

우리의 Model Class 내에서 Model이 좋은지/나쁜지 판단할 수 있는 기준인 손실함수를 정의한다.

[3] Pick Optimizer

정의한 손실함수를 줄이기 위해 θ 를 어떻게 업데이트할지 결정한다.

Machine Learning method

[1] Define Model Class

[2] Define Loss Function

[3] Pick Optimizer

Model Class

Model Class : 학습 파라미터 θ 를 가진 함수들의 집합
= 가능한 함수 후보의 범위

Model Class : 가능한 함수 후보 집합

Parameter : 후보 중 하나를 선택하는 값

Training : 좋은 파라미터를 찾는 과정

Model Class

$$: f_{\theta}(x) = \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

$$f_{\theta}(x) = x_1 + 2x_2 + 3x_3$$

$$f_{\theta}(x) = 2x_1 - x_2 + 5x_3$$

$$f_{\theta}(x) = 100x_1 + 30x_3$$

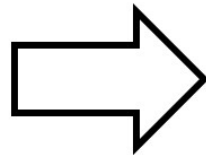
앞으로 다항함수 모델을 사용할 예정

Supervised Learning

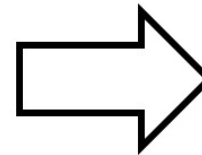
Given : $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

Learn model : $f_{\theta}(x) \approx y$

Given : Dataset



Model
 $f_{\theta}(x)$



Predict : Label

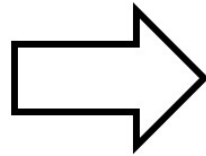
Dog 0
Cat X

Supervised Learning

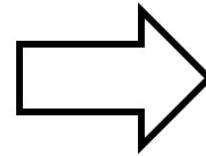
Given : $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

Learn model : $f_{\theta}(x) \approx y$

Given : Dataset



Model
 $f_{\theta}(x)$



Predict : Label

Dog ?

Cat ?

Label 예측이 아닌
Class별 확률을 예측하자!

Supervised Learning

Predicting Probability

Label Prediction

 5

 9

Probability Prediction

 3: 10% 5: 80% 6: 10%

 0: 4% 4: 11% 8: 6% 9: 75%

Pros

1. 확률이 모델의 불확실성을 잘 표현한다.

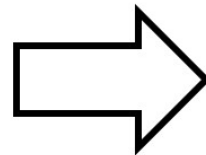
2. 확률은 조금씩 변화가 가능하므로 학습 신호가 더 부드럽다. (probability는 연속적으로 변화 가능)

 3: 10% 5: 80% 6: 10% → 3: 10% 5: 40% 6: 50%

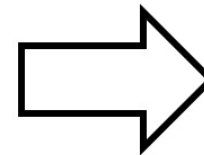
Supervised Learning

Predicting Probability

Given : Dataset



Model



Before : $f_{\theta}(x) \approx y$

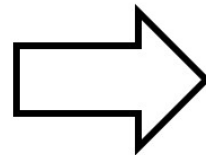
After : $P_{\theta}(y|x)$

Cat : 40%
Dog : 50%
Human : 10%

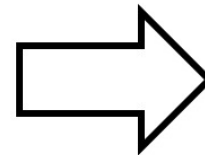
Supervised Learning

Predicting Probability

Given : Dataset



Model



Before : $f_{\theta}(x) \approx y$

After : $P_{\theta}(y|x)$

Cat : 40%
Dog : 50%
Human : 10%

How can we represent probability?

Supervised Learning

Predicting Probability

확률의 조건

1. Positive : $\mathbb{R} \rightarrow \mathbb{R}^+$

Ex. z^2 $|z|$ $\max(0, z)$ e^z

2. Sum to 1 : $\int p(x) dx = 1$

모든 값을 더해서 나누자 : normalize

분모 : $\int z dz$

Supervised Learning

Predicting Probability

왜 지수함수를 주로 택하는 것일까?

확률의 조건

1. Positive : $\mathbb{R} \rightarrow \mathbb{R}^+$

Ex. z^2 $|z|$ $\max(0, z)$

e^z

1. 확률의 큰 변화를 표현하기에 좋다.

2. Exponential Family(Normal, Gamma, Bernoulli, Poisson ...)를 이용할 수 있다.

2. Sum to 1 : $\int p(x) dx = 1$

모든 값을 더해서 나누자 : normalize
분모 : $\int z dz$

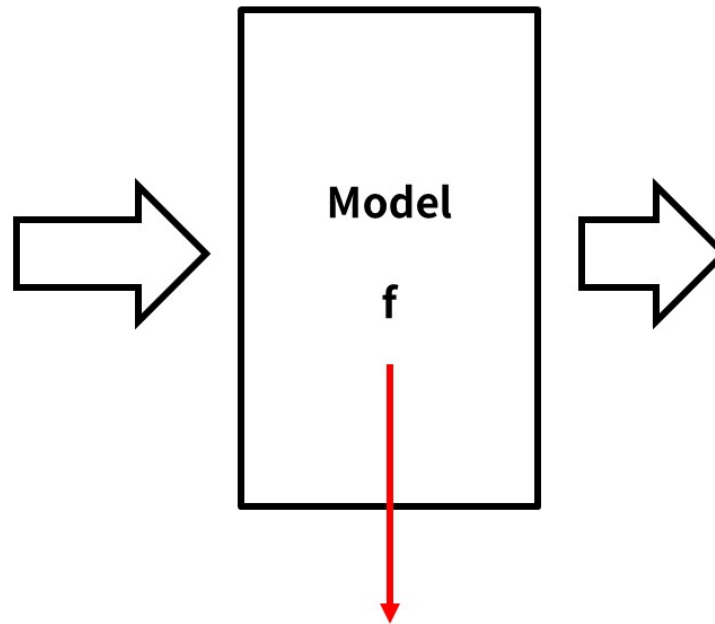
$softmax(z_1, z_2, \dots, z_n)$

$$= \frac{e^{z_i}}{e^{z_1} + e^{z_2} + \dots + e^{z_n}}$$

Supervised Learning

Predicting Probability

Given : Dataset



모델이 label 만큼의 벡터를 반환하면
확률 벡터로 변환

$$f_{dog}(x) = x^T \cdot \theta_{dog}$$
$$f_{cat}(x) = x^T \cdot \theta_{cat}$$

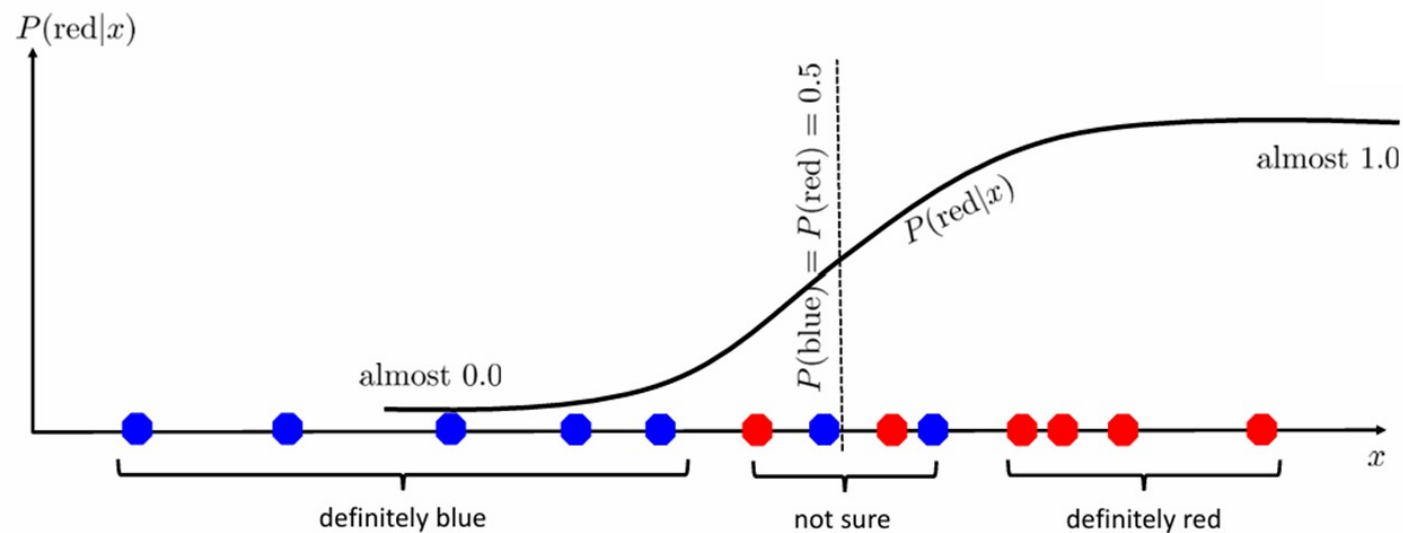
$$p(y|x) = \text{softmax}(f_{dog}(x), f_{cat}(x))$$
$$= \frac{e^{f_{dog}(x)}}{e^{f_{cat}(x)} + e^{f_{dog}(x)}}$$

어떤 종류의 함수도 가능
: 충분히 유연한 함수를 정하고 그 안에서 최적 파라미터를 찾는다.

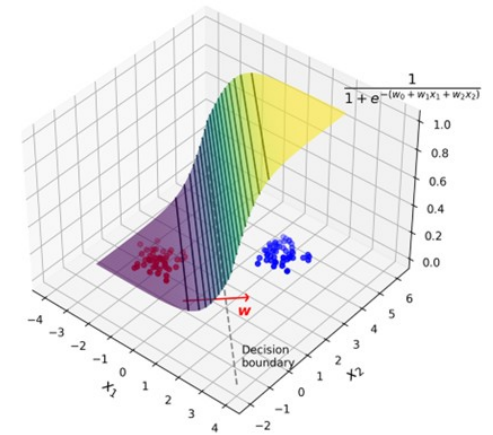
Supervised Learning

Softmax

클래스 i의 logit을 확률분포로 변환해주는 함수



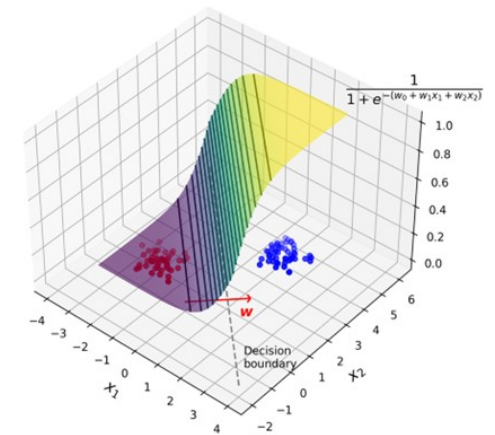
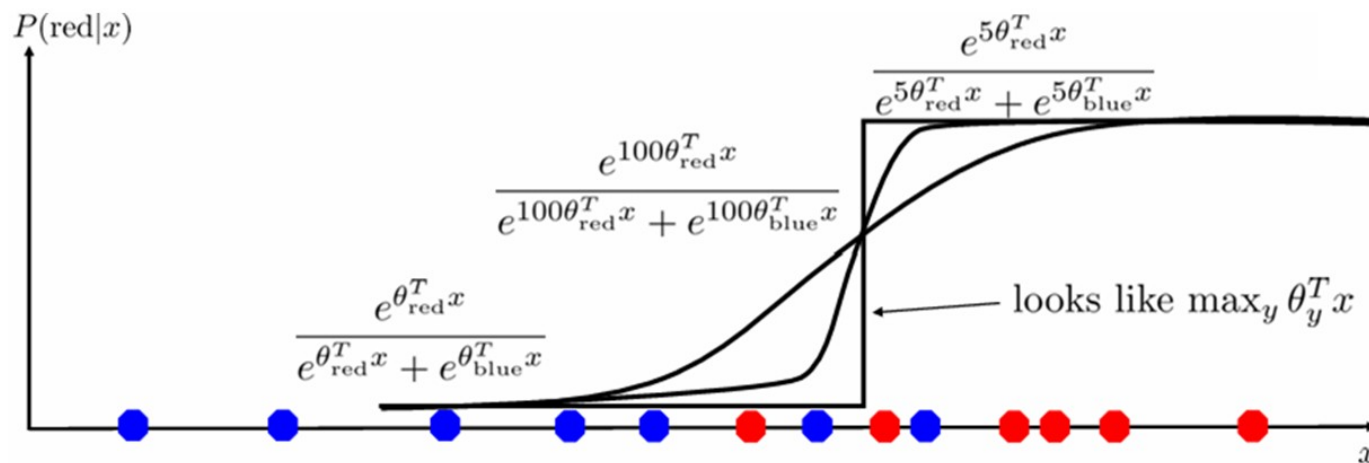
Soft :
결정경계 근처에서 확률이 부드럽게 변한다.



Supervised Learning

Softmax

클래스 i의 logit을 확률분포로 변환해주는 함수



Max :

가장 큰 확률을 가진 클래스를 선택하는 max 함수와 비슷한 역할을 한다.

Machine Learning method

[1] Define Model Class

[2] Define Loss Function

[3] Pick Optimizer

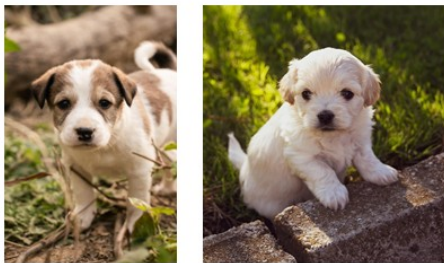
Loss function

How is the dataset “generated”?

Real World



Sampling



Key assumption in sampling

- Independent : 각각의 샘플은 서로 독립적
- Identically distributed : 각각의 샘플은 똑같은 분포에서 추출

Dataset $p(D) = \prod_i p(x_i, y_i) = \prod_i p(x_i)p(y_i|x_i)$

Dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

Loss function

How is the dataset “generated”?

Dataset $p(D) = \prod_i p(x_i, y_i) = \prod_i p(x_i) p(y_i|x_i)$

Supervised Learning : x 가 주어졌을 때 y 예측 목표

$$p_{\theta}(y_i|x_i) \approx p(y_i|x_i)$$

Model distribution $\approx$ True distribution

→ Maximize $\prod_i p(x_i) p_{\theta}(y_i|x_i)$

관측된 Dataset에 대해서는 모든 sample에 대해 실제 관측된 label의 확률이 커야 한다.

Loss function

How is the dataset “generated”?

Maximize $\prod_i p(x_i)p_\theta(y_i|x_i)$

Problem : 1 이하의 숫자를 계속해서 곱하게 되어 값이 작아지게 된다.

Solution : Log를 취해 곱을 합으로 바꾼다.

Maximize $\log p(D) = \sum_i \log p(x_i)P_\theta(y_i|x_i)$

$$= \sum_i \log P(x_i) + \log P_\theta(y_i|x_i)$$

Maximize likelihood

$$= \sum_i \log P_\theta(y_i|x_i) + \text{const}$$

Loss function

How is the dataset “generated”?

Maximize $\prod_i p(x_i)p_\theta(y_i|x_i)$

Problem : 1 이하의 숫자를 계속해서 곱하게 되어 값이 작아지게 된다.

Solution : Log를 취해 곱을 합으로 바꾼다.

$$\begin{aligned}\text{Maximize } \log p(D) &= \sum_i \log p(x_i)P_\theta(y_i|x_i) \\ &= \sum_i \log P(x_i) + \log P_\theta(y_i|x_i)\end{aligned}$$

$$= \sum_i \log P_\theta(y_i|x_i) + \text{const}$$



Maximize likelihood

$$\theta^* \leftarrow \arg \max_\theta \sum_i \log P_\theta(y_i|x_i)$$

$$\theta^* \leftarrow \arg \min_\theta \underbrace{- \sum_i \log P_\theta(y_i|x_i)}$$

Negative Log-Likelihood

Loss function

Example of Loss functions

Loss : 현재 모델이 관측된 데이터에 대해 얼마나 나쁜지 진단하는 정량적 지표

$$-\sum_i \log P_{\theta}(y_i|x_i) \quad \text{Negative log-likelihood}$$

$$\sum_i \delta(f_{\theta}(x_i) \neq y_i) \quad \text{Zero-one loss}$$

$$\sum_i \frac{1}{2} \|f_{\theta}(x_i) - y_i\|^2 \quad \text{Mean-squared error}$$

Case Study

Logistic Regression

1. Select Model

$$f_{\theta}(x) = x^T \theta$$

$$\begin{aligned} & \text{softmax}(z_1, z_2, \dots, z_n) \\ &= \frac{e^{z_i}}{e^{z_1} + e^{z_2} + \dots + e^{z_n}} \end{aligned}$$

2. Define Loss function

$$L(\theta) = - \sum_{i=1}^n \log_{p_{\theta}}(y_i | x_i)$$

3. Optimization Method

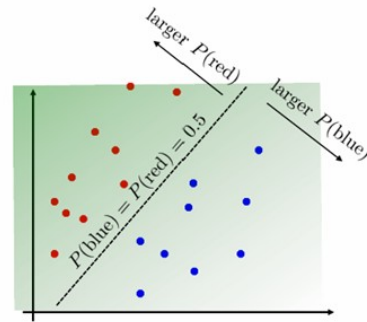
Gradient Descent

Case Study

Logistic Regression for Binary Classification

1. Select Model

$$f_{\theta}(x) = x^T \theta$$



2. Define Loss function

$$L(\theta) = - \sum_{i=1}^n \log_{P\theta}(y_i|x_i)$$

3. Optimization Method

Gradient Descent

$$p(z_1) = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} \quad \text{양변을 } e^{z_1} \text{ 로 나눈다.}$$

$$= \frac{1}{1 + e^{z_2 - z_1}}$$

$$= \frac{1}{1 + e^z}$$

파라미터의 개수를 1개로 줄일 수 있다.

Risk

Empirical Risk vs True Risk

$$\text{Risk} : E_{x \sim p(x), y \sim p(y|x)}[L(x, y, \theta)]$$

모든 데이터를 고려했을 때 모델의 평균 손실

일반적으로 두 Risk는 비슷하다.

$$\text{Empirical Risk} : \frac{1}{n} \sum_{i=1}^n L(x_i, y_i, \theta) \approx E_{x \sim p(x), y \sim p(y|x)}[L(x, y, \theta)]$$

훈련 데이터 위에서의 모델의 평균 손실

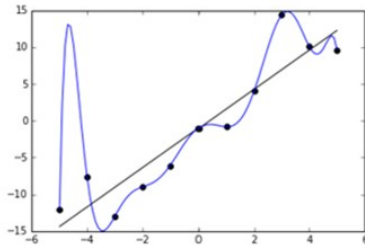
Risk

Empirical Risk Minimization

$$\frac{1}{n} \sum_{i=1}^n L(x_i, y_i, \theta) \approx E_{x \sim p(x), y \sim p(y|x)} [L(x, y, \theta)]$$

But Empirical risk minimization is **not always same** as true risk minimization

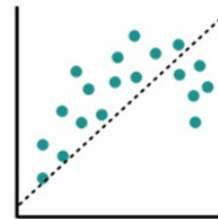
Case 1. Overfitting



Can happen

1. Dataset is too small
2. Model is too powerful

Case 2. Underfitting



Can happen

1. Model is too weak
2. Optimizer is not configured well

Risk

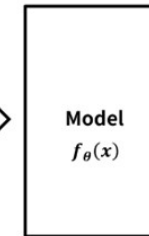
Analyzing Error

Suppose we picked **regression model**
+ 정답 y 출력이 정규분포를 따른다고 가정



Given : Dataset

Height : 180cm
Weight : 75kg



Predict : Life Span

75 yr

$$\log P_{\theta}(y|x) = \log N(f_{\theta}(x), \Sigma_{\theta}(x))$$

$$= -\frac{1}{2}(f_{\theta}(x) - y)^T \Sigma_{\theta}^{-1}(x)(f_{\theta}(x) - y) - \frac{1}{2} \log |\Sigma_{\theta}(x)| + \text{const}$$

$$\text{if } \Sigma_{\theta}(x) = I$$

$$= -\frac{1}{2}(f_{\theta}(x) - y)^T (f_{\theta}(x) - y) - \frac{1}{2} \log 1 + \text{const}$$

$$= -\frac{1}{2} \|f_{\theta}(x) - y\|^2 + \text{const} \quad \text{Same as MSE Loss}$$

Risk

Analyzing Error

Dataset이 변할 때 이 Loss가 어떻게 변하는지 분석하자.

$$E_{D \sim P(D)} [\|f_\theta(x) - y\|^2] = E_{D \sim P(D)} [\|f_D(x) - f(x)\|^2]$$

$$= \sum_D p(D) \|f_D(x) - f(x)\|^2$$

Definition of expectation

$$= E_{D \sim P(D)} [\|f_D(x) - \bar{f}(x)\|^2] + E_{D \sim P(D)} [\|\bar{f}(x) - f(x)\|^2]$$

Variance

데이터셋이 바뀔 때마다
모델 예측과 평균과의 차이

Bias^2

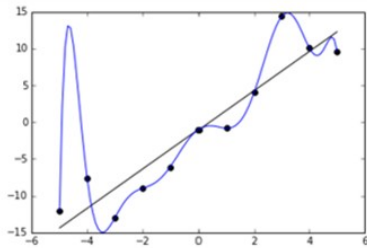
여러 데이터셋에 대해
평균적으로 학습된 함수와 실제 함수와의 차이

Risk

Bias-Variance Tradeoff

$$\underbrace{E_{D \sim P(D)} [\|f_D(x) - \bar{f}(x)\|^2]}_{\text{Variance}} + \underbrace{E_{D \sim P(D)} [\|\bar{f}(x) - f(x)\|^2]}_{\text{Bias}^2}$$

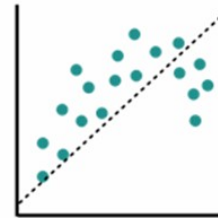
Case 1. Overfitting : **High variance**



Can happen when

1. Dataset is too small
2. Model is too powerful

Case 2. Underfitting : **High Bias**



Can happen when

1. Model is too weak
2. Optimizer is not configured well

Regularization

How to regulate bias/variance?

$$\underbrace{E_{D \sim P(D)} [\|f_D(x) - \bar{f}(x)\|^2]}_{\text{Variance}} + \underbrace{E_{D \sim P(D)} [\|\bar{f}(x) - f(x)\|^2]}_{\text{Bias}^2}$$

1. Get more data → Variance 해결 가능, Bias 해결 불가능
2. Change model class → 불연속적으로 규제됨
3. **Regularization term** → 연속적으로 Bias/Variance 규제 가능

Regularization

Bayesian perspective

$$p(\theta|D) = \frac{P(\theta, D)}{p(D)} \propto p(\theta, D) = \underbrace{P(D|\theta)}_{\text{Dataset을 보기 전 } \theta \text{가 얼마나 그럴듯한지 설명}} \underbrace{p(\theta)}_{\text{Ex. 작은 } \theta \text{값이 더 그럴듯하다} \rightarrow p(\theta) \uparrow}$$

모델이 정해졌을 때 Dataset의 확률

$$= \prod_i p(x_i)p_{\theta}(y_i|x_i)$$

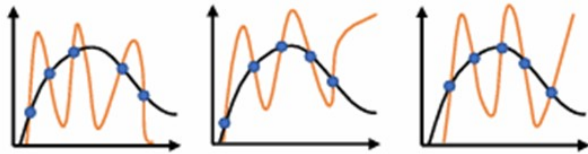
$$\text{New Loss} = -(\sum_i \log P_{\theta}(y_i|x_i)) - \log p(\theta)$$

Regularization

Regularized linear regression

$$p(\theta|D) = \frac{P(\theta, D)}{p(D)} \propto p(\theta, D) = P(D|\theta) p(\theta)$$

$$\text{New Loss} = -(\sum_i \log P_{\theta}(y_i|x_i)) - \log p(\theta)$$



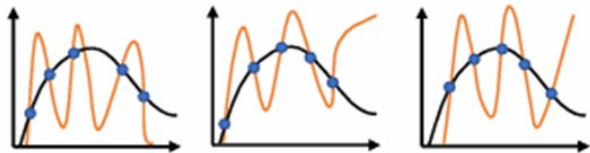
특징 : θ 가 높은 값이다. $\rightarrow$ θ 가 작은 값을 가질 확률을 높게 부여하자

Regularization

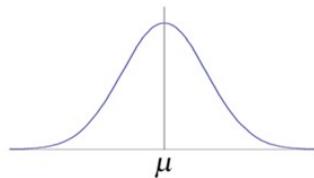
Regularized linear regression

$$p(\theta|D) = \frac{P(\theta, D)}{p(D)} \propto p(\theta, D) = P(D|\theta) p(\theta)$$

$$\text{New Loss} = -(\sum_i \log P_\theta(y_i|x_i)) - \log p(\theta)$$



특징 : θ 가 높은 값이다. $\rightarrow$ θ 가 작은 값을 가질 확률을 높게 부여하자



Idea : $p(\theta)$ 를 $N(0, \sigma^2)$ 으로 설정하자!

Regularization

Regularized linear regression

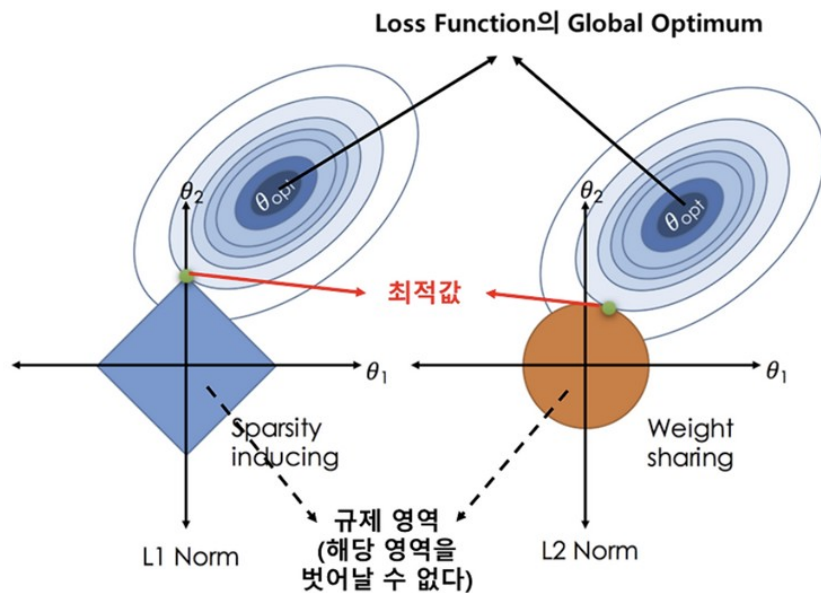
$$p(\theta|D) = \frac{P(\theta, D)}{p(D)} \propto p(\theta, D) = P(D|\theta) p(\theta)$$

$$\begin{aligned} \text{New Loss} &= -(\sum_i \log P_\theta(y_i|x_i)) - \log p(\theta) \\ &= \sum_i^D -\frac{1}{2} \frac{\theta_i^2}{\sigma^2} - \log \sigma - \frac{1}{2} \log 2\pi = -\lambda \|\theta\|^2 \end{aligned}$$

$$= -\frac{1}{2} \|f_\theta(x) - y\|^2 - \lambda \|\theta\|^2$$

Regularization

Examples of regularizers



1. L1 Regularization $-\lambda|\theta|$

2. L2 Regularization $-\lambda\|\theta\|^2$

3. Dropout 신경망에서 학습 중 일부 뉴런 랜덤하게 off

$$\tilde{h} = h \cdot m, \quad h = [h_1, h_2, h_3, h_4], \quad m = [1, 0, 1, 0]$$
$$\tilde{h} = [h_1, 0, h_3, 0]$$

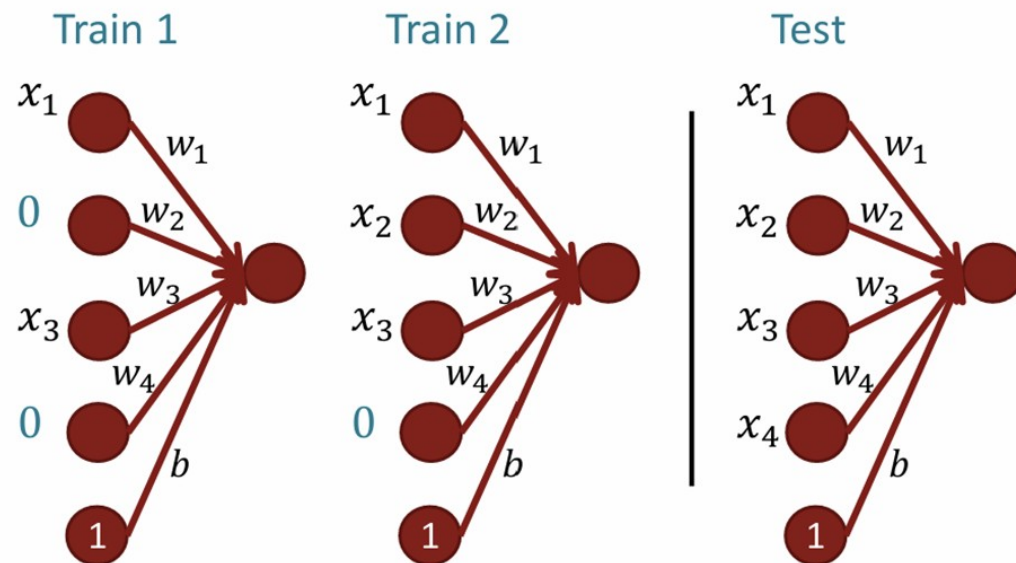
4. Gradient penalty

Gradient가 너무 크거나 불안정해지는 것 방지

$$+\lambda\|\nabla f_{\theta}(x)\|^2$$

Regularization

Dropout



$$\tilde{h} = h \cdot m,$$
$$\tilde{h} = [h_1, 0, h_3, 0]$$

$$h = [h_1, h_2, h_3, h_4], \quad m = [1, 0, 1, 0]$$

Training :

For each data point each time:

- Randomly set input to 0 with prob : p

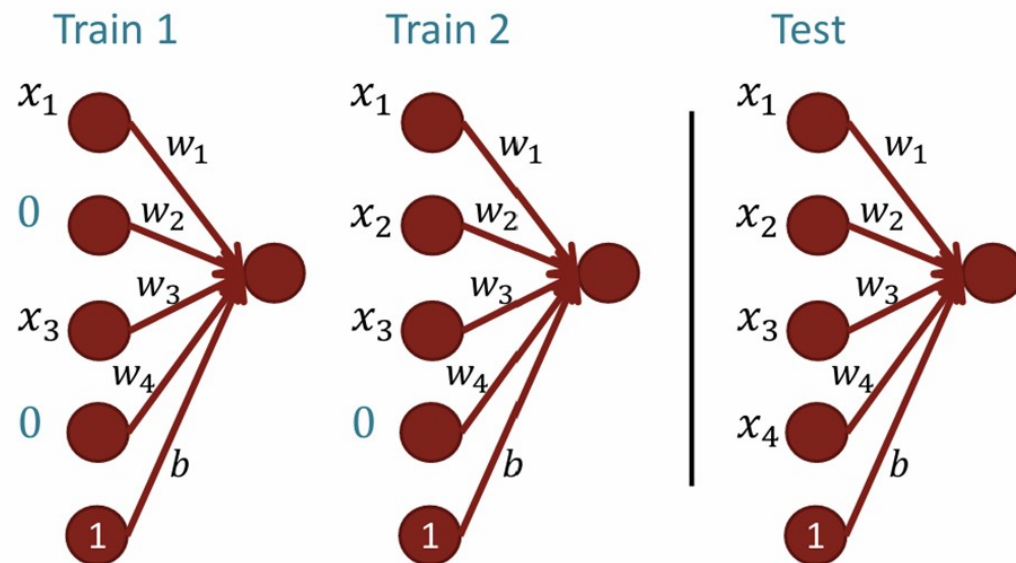
Testing :

Multiply all weights by $1-p$

$$(y = (1 - p)(w_1x_1 + w_2x_2 + \dots + w_nx_n))$$

Regularization

Dropout



$$\tilde{h} = h \cdot m,$$
$$\tilde{h} = [h_1, 0, h_3, 0]$$

$$h = [h_1, h_2, h_3, h_4], \quad m = [1, 0, 1, 0]$$

Training :

For each data point each time:

- Randomly set input to 0 with prob : p
dropout ratio
Often $p = 0.5$ / $p=0.15$ for input layer

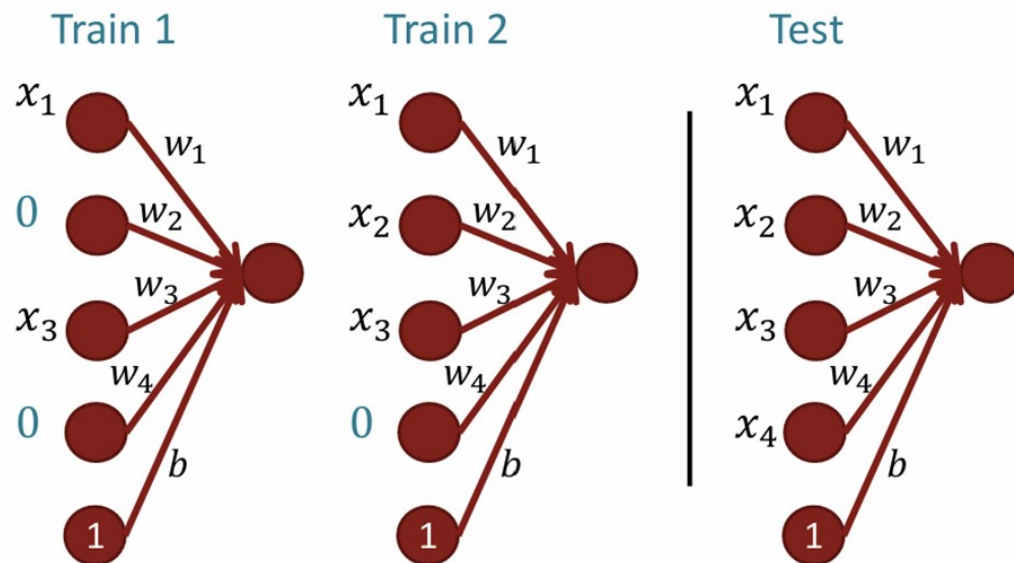
Testing :

Multiply all weights by $1-p$

$$(y = (1 - p)(w_1x_1 + w_2x_2 + \dots + w_nx_n))$$

Regularization

Dropout

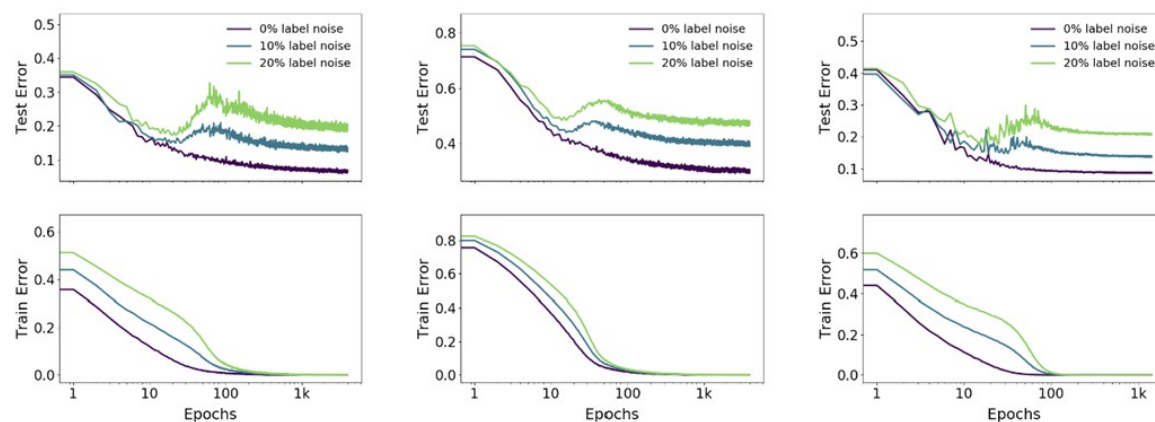


$$\tilde{x} = m \otimes x$$
$$y = (1 - p)(w_1x_1 + w_2x_2 + \dots + w_nx_n)$$

$$\tilde{x} = \frac{m \otimes x}{1 - p}$$
$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

More about Overfitting

Double descent



(a) ResNet18 on CIFAR10.

(b) ResNet18 on CIFAR100.

(c) 5-layer CNN on CIFAR 10.

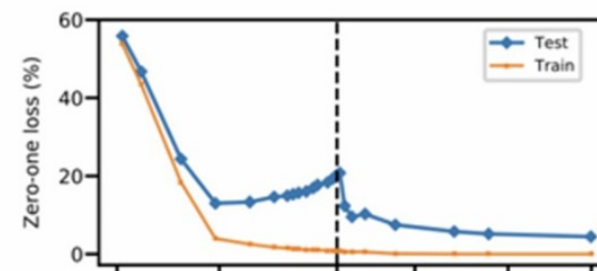
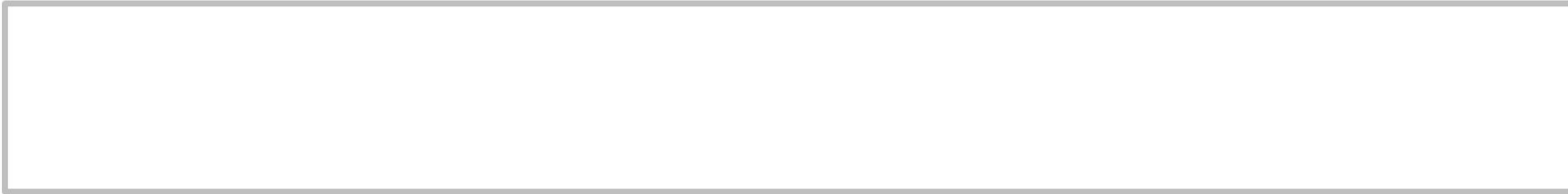


Figure 10: **Epoch-wise double descent** for ResNet18 and CNN (width=128). ResNets trained using Adam with learning rate 0.0001, and CNNs trained with SGD with inverse-squareroot learning rate.

For large model, regularization produces models that generalize well

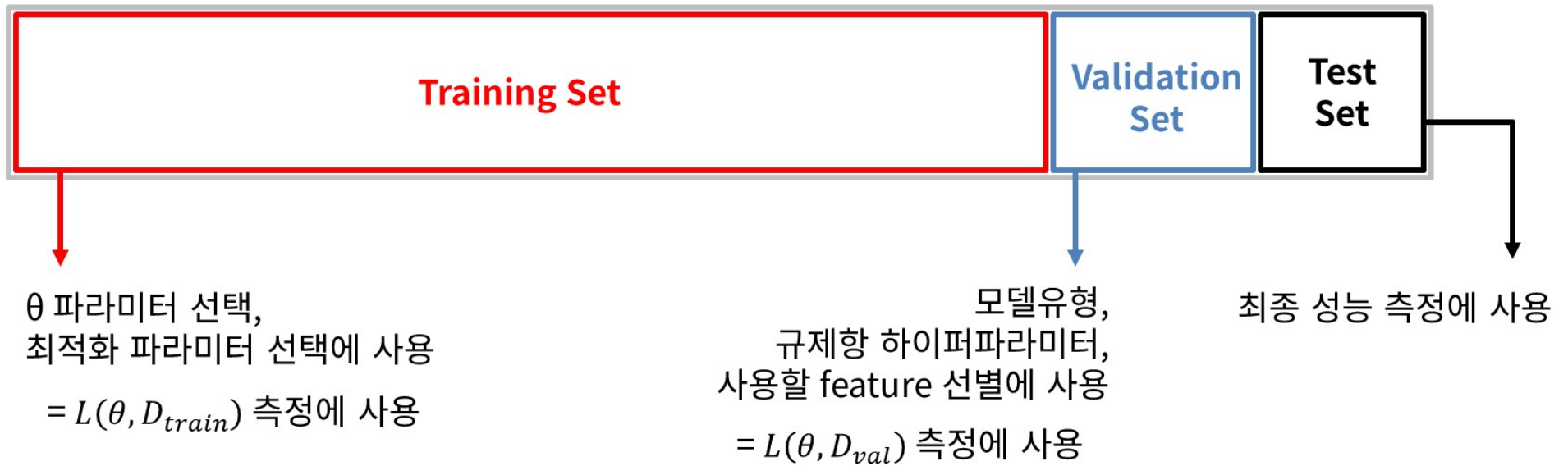
| Data Split

Dataset

A large, empty rectangular box with a thin gray border, intended for inputting a dataset name or path.

Data Split

Dataset



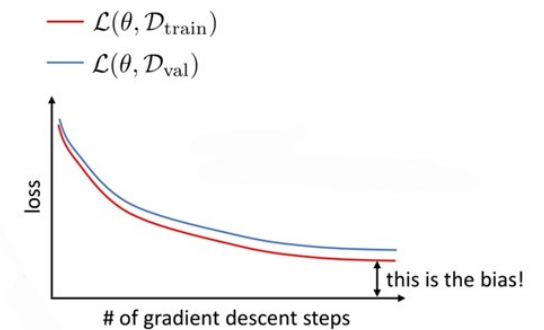
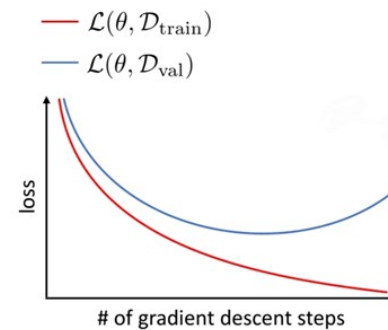
Data Split

Dataset



Case 1. $L(\theta, D_{train})$ is not low enough $\rightarrow$ underfitting
- decrease regularization

Case 2. $L(\theta, D_{train}) \ll L(\theta, D_{val}) \rightarrow$ overfitting
- increase regularization



Machine Learning method

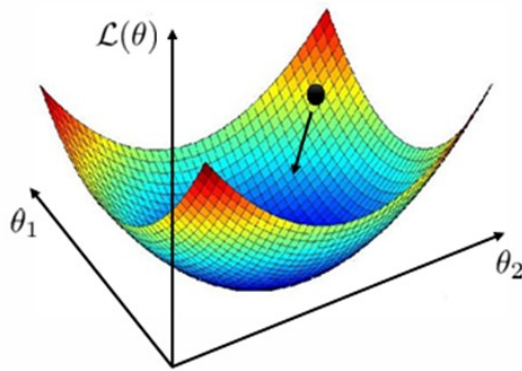
[1] Define Model Class

[2] Define Loss Function

[3] Pick Optimizer

Optimization

Gradient Descent

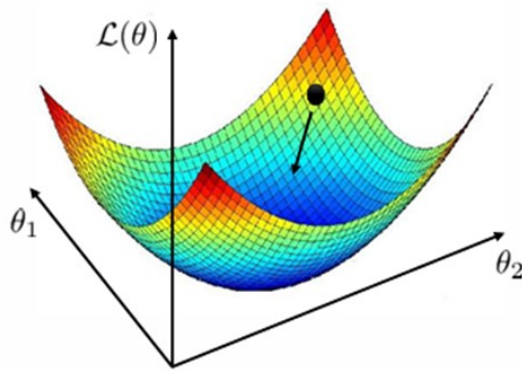


알고리즘

1. Loss가 감소하는 방향 v 를 찾는다.
2. $\theta \leftarrow \theta + \alpha v$

Optimization

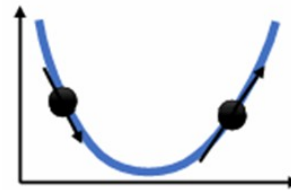
Gradient Descent



알고리즘

1. Loss가 감소하는 방향 v 를 찾는다.

2. $\theta \leftarrow \theta + \alpha v$

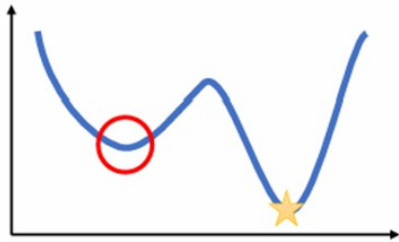


Loss가 감소하는 방향은 slope의 반대방향이다.

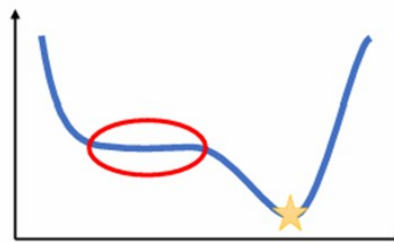
$$v = -\nabla_{\theta} L(\theta)$$

Optimization

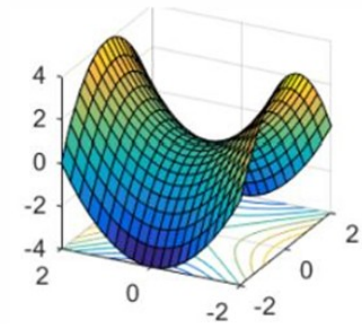
3 Problem in loss landscape



Local Optima



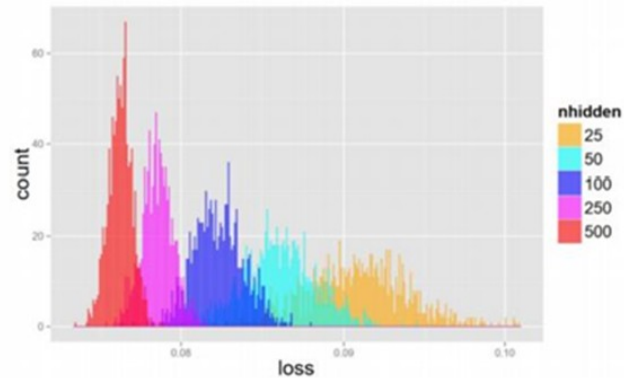
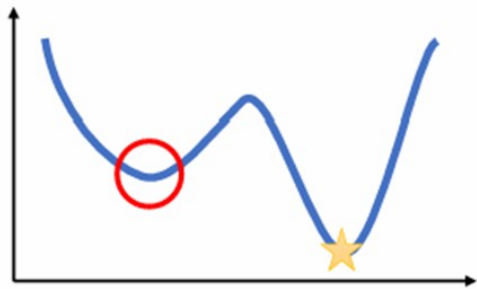
Plateau



Saddle Point

Optimization

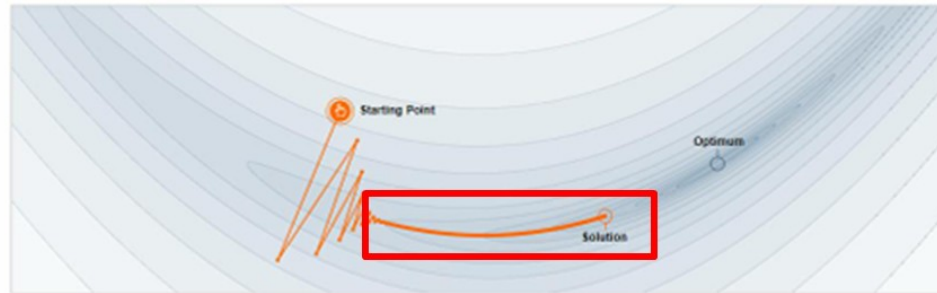
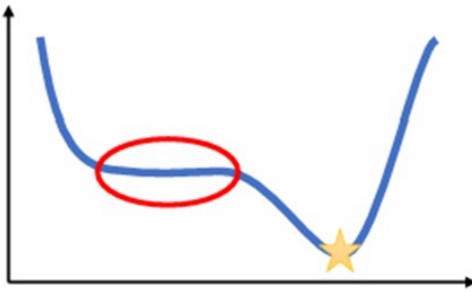
Local Optima



해가 국소최적해에 갇혀 최적해를 찾지 못한다.
하지만, 큰 모델에서는 큰 영향이 없다.

Optimization

Plateaus

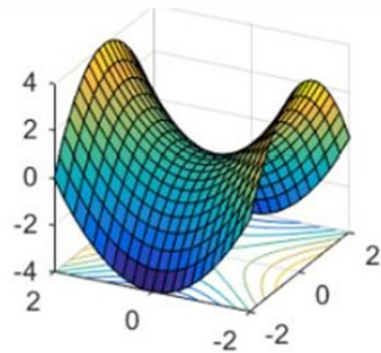


기울기가 매우 작은 지형

무한의 시간이 주어질 경우 문제가 되지는 않으나,
유한한 시간에 해당 구간을 빠르게 통과하기 위해서 큰 학습률이 필요하다.

Optimization

Saddle Point



$$\left[\begin{array}{cc} 1 & 0 \\ 0 & -1 \end{array} \right]$$

*Saddle 판별 방법:

해당 위치에서 Hessian 연산

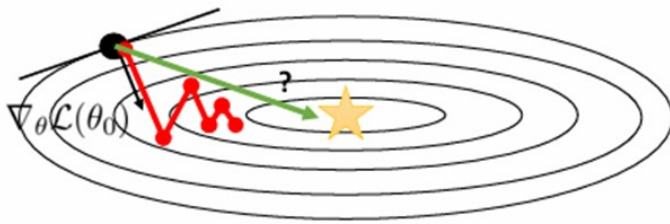
- 모든 고유값 > 0 : local minimum
- 모든 고유값 < 0 : local maximum
- 양수 고유값, 음수 고유값 혼재 : saddle

기울기가 매우 작은 지형

어떤 차원에서는 local minimum, 어떤 차원에서는 local maximum이다.

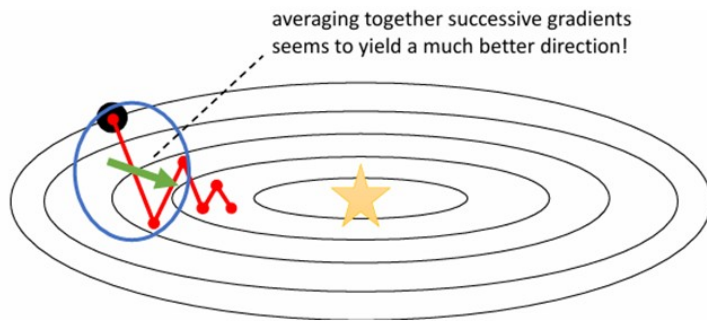
Optimization

문제점 : 음의 Gradient 방향은 항상 최적의 방향은 아니다.



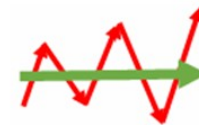
$v = -\nabla_{\theta} L(\theta)$: steepest direction

Momentum



과거의 방향까지 반영하여 direction을 결정한다.

Case 1. 과거의 이동방향이 현재 이동방향과 반대인 경우



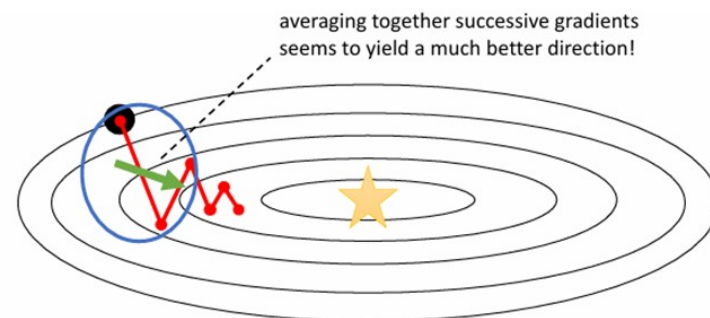
Case 2. 과거의 이동방향이 현재 이동방향과 같은 경우



Momentum

$$\theta_{k+1} = \theta_k - \alpha g_k \quad g_k = \nabla_{\theta} L(\theta_k) + \mu g_{k-1}$$

$$\begin{aligned} g_k &= \nabla_{\theta} L(\theta_k) + \mu g_{k-1} \\ &= \nabla_{\theta} L(\theta_k) + \mu \nabla_{\theta} L(\theta_{k-1}) + \mu^2 g_{k-2} \\ &= \dots \\ &= \nabla_{\theta} L(\theta_k) + \mu \nabla_{\theta} L(\theta_{k-1}) + \dots + \mu^k \nabla_{\theta} L(\theta_0) \end{aligned}$$



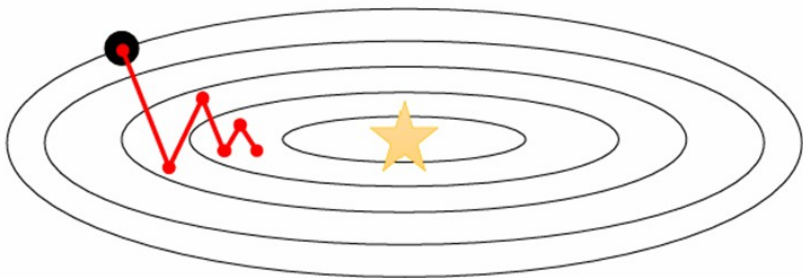
Momentum 계수가 커지면 과거 방향 정보를 잘 보존한다.
따라서 적절한 값에서는 최적점까지 더 빠르게 도달할 수 있다.

다만 너무 크면 진동하거나 최적점을 지나칠 수 있다.

Gradient Scale

Gradient의 방향은 유용하지만, 각 차원별 gradient scale은 서로 크게 다를 수 있다.

이 점에서 최적 방향은 세로축 방향보다 가로축 방향 이동이 필요하다.

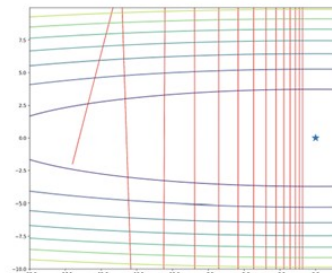


$$L(\theta) = \|f_{\theta}(x) - y\|^2$$
$$\nabla_{\theta} L(\theta) = 2(f_{\theta}(x) - y)^T \frac{df}{d\theta}$$

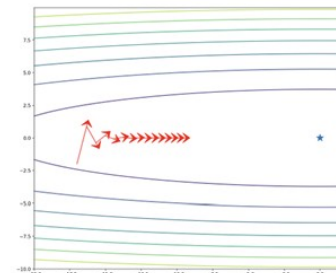
모든 차원에 같은 learning rate 적용 시 문제 발생 가능

Optimum에서 먼 경우 $\nabla_{\theta} L(\theta)$ 값이 커서 작은 학습률 필요
Optimum에서 가까운 경우 $\nabla_{\theta} L(\theta)$ 값이 작아서 큰 학습률 필요

Gradient의 각 차원을 normalize 하자!



(b) With a large learning rate, GD on this ill-conditioned problem oscillates and diverges in the vertical direction.



(c) With a small learning rate, GD on this ill-conditioned problem remains stable, but makes very slow progress towards the optimum.

Algorithm

RMSProp

Estimate per-dimension magnitude

$$s_k \leftarrow \beta s_{k-1} + (1 - \beta)(\nabla_{\theta} L(\theta_k))^2$$

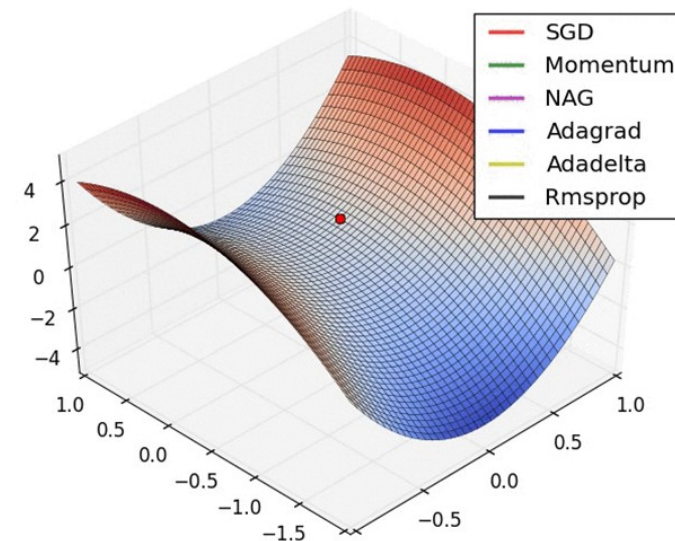
$$\theta_{k+1} = \theta_k - \alpha \frac{\nabla_{\theta} L(\theta_k)}{\sqrt{s_k}}$$

AdaGrad

Estimate per-dimension **cumulative** magnitude

$$s_k \leftarrow s_{k-1} + (\nabla_{\theta} L(\theta_k))^2$$

$$\theta_{k+1} = \theta_k - \alpha \frac{\nabla_{\theta} L(\theta_k)}{\sqrt{s_k}}$$



AdaGrad는 s_k 를 계속 누적한다.
→ Convex problem에서는 좋은 솔루션을
제공하지만 대부분의 경우 RMSprop이 좋다.

Algorithm

Adam

Combining momentum and RMSprop

$$m_k \leftarrow (1 - \beta_1)\nabla_{\theta}L(\theta_k) + \beta_1 m_{k-1} \quad \text{Gradient의 이동평균}$$

$$v_k \leftarrow (1 - \beta_2)(\nabla_{\theta}L(\theta_k))^2 + \beta_2 v_{k-1} \quad \text{Gradient 제곱에 대한 이동평균}$$

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k} \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

$$\theta_{k+1} = \theta_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}$$

$$m_0 = 0$$

$$m_1 = (1 - \beta_1)\nabla_{\theta}L(\theta_1)$$

$$m_2 = (1 - \beta_1)\nabla_{\theta}L(\theta_2) + \beta_1(1 - \beta_1)\nabla_{\theta}L(\theta_1)$$

$$\begin{aligned} m_k = & (1 - \beta_1)\nabla_{\theta}L(\theta_k) + \beta_1(1 - \beta_1)\nabla_{\theta}L(\theta_{k-1}) \\ & + \beta_1^2(1 - \beta_1)\nabla_{\theta}L(\theta_{k-2}) \\ & + \dots + \beta_1^{k-1}(1 - \beta_1)\nabla_{\theta}L(\theta_1) \end{aligned}$$

Stochastic Optimization

SGD(Stochastic Gradient Descent)

Minibatch로 Gradient를 추정하는 방식

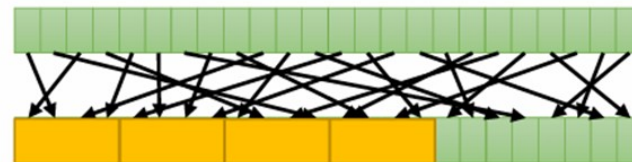
$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(y_i|x_i) \approx -E_{p_{\text{data}}(x,y)}[\log p_{\theta}(y_i|x_i)] \approx -\frac{1}{B} \sum_{j=1}^B \log p_{\theta}(y_{i_j}|x_{i_j})$$

Problem : 매 단계마다 모든 데이터포인트에 대해 계산하려면 연산량이 많다.

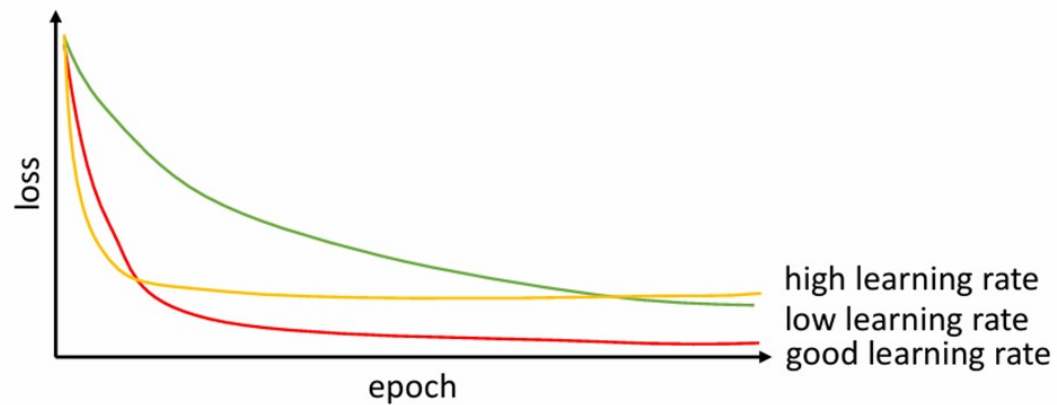
Solution : 데이터셋에서 **작은 크기의 Batch**를 뽑아서 **Batch에 대해서 업데이트**하자

과정

1. 매 단계마다 Minibatch를 뽑는다.
2. Minibatch에 대해서 Loss 계산하여 업데이트한다.

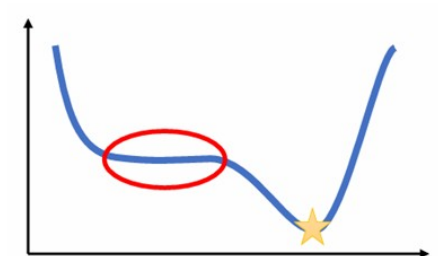


Decaying Learning Rate

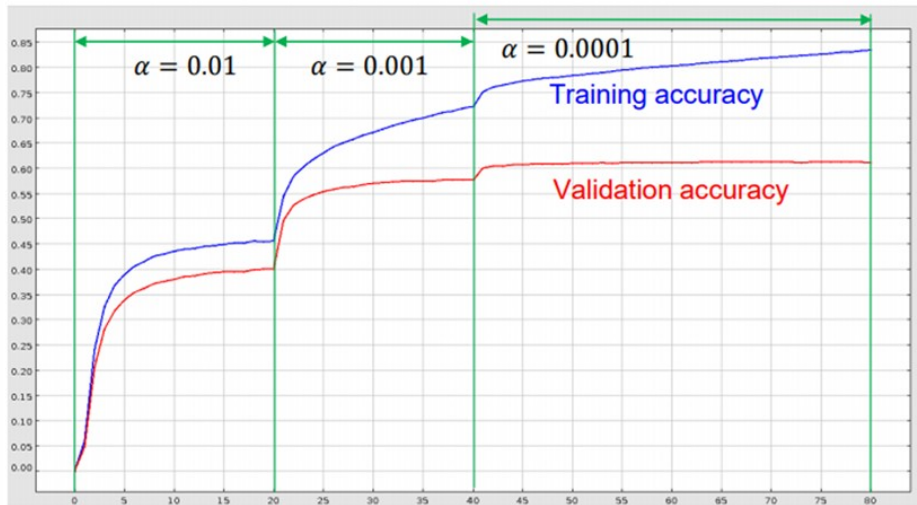


High learning rate : 최적점을 지나칠 수 있다.

Low learning rate : 학습 시간이 유한한 경우 low gradient area에 갇힐 수 있다.



Decaying Learning Rate



학습이 진행됨에 따라 학습률을 낮추는 방법

Validation loss가 더 이상 개선되지 않을 때 learning rate를 낮춘다.

| Quiz

Quiz

Q1. Logistic Regression

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \theta_{\text{dog}} = \begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix} \quad \theta_{\text{cat}} = \begin{bmatrix} 0.6 \\ 0.1 \end{bmatrix}$$

$$z = \theta^T x$$

$$e^{1.0} \approx 2.72$$

$$e^{0.8} \approx 2.23$$

Q1-1. $z_{\text{dog}}, z_{\text{cat}}$ 을 계산하시오.

Q1-2. $p(y = \text{dog}|x), p(y = \text{cat}|x)$ 를 계산하시오.

Q1-3. 모델의 최종 예측 label을 구하시오.

Q1-4. 실제 label이 Dog일 때 NLL Loss를 구하시오.

Quiz

Q1. Logistic Regression

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \theta_{\text{dog}} = \begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix} \quad \theta_{\text{cat}} = \begin{bmatrix} 0.6 \\ 0.1 \end{bmatrix} \quad z = \theta^T x$$

Q1-1. z_{dog} , z_{cat} 을 계산하시오.

$$z_{\text{dog}} = \theta_{\text{dog}}^T x = 0.4 \times 1 + 0.3 \times 2 = 1.0$$

$$z_{\text{cat}} = \theta_{\text{cat}}^T x = 0.6 \times 1 + 0.1 \times 2 = 0.8$$

Quiz

Q1. Logistic Regression

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \theta_{\text{dog}} = \begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix} \quad \theta_{\text{cat}} = \begin{bmatrix} 0.6 \\ 0.1 \end{bmatrix} \quad z = \theta^T x$$

Q1-2. $p(y = \text{dog}|x), p(y = \text{cat}|x)$ 를 계산하시오.

$$z_{\text{dog}} = \theta_{\text{dog}}^T x = 0.4 \times 1 + 0.3 \times 2 = 1.0$$

$$z_{\text{cat}} = \theta_{\text{cat}}^T x = 0.6 \times 1 + 0.1 \times 2 = 0.8$$

$$p(y = \text{dog}|x) = \frac{e^{1.0}}{e^{1.0} + e^{0.8}} = \frac{2.72}{2.72 + 2.23} \approx 0.55$$

$$p(y = \text{cat}|x) = \frac{e^{0.8}}{e^{1.0} + e^{0.8}} = \frac{2.23}{2.72 + 2.23} \approx 0.45$$

Quiz

Q1. Logistic Regression

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \theta_{\text{dog}} = \begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix} \quad \theta_{\text{cat}} = \begin{bmatrix} 0.6 \\ 0.1 \end{bmatrix} \quad z = \theta^T x$$

Q1-3. 모델의 최종 예측 label을 구하시오.

$$z_{\text{dog}} = \theta_{\text{dog}}^T x = 0.4 \times 1 + 0.3 \times 2 = 1.0$$

$$z_{\text{cat}} = \theta_{\text{cat}}^T x = 0.6 \times 1 + 0.1 \times 2 = 0.8$$

$$p(y = \text{dog} | x) = \frac{e^{1.0}}{e^{1.0} + e^{0.8}} \approx \frac{2.72}{2.72 + 2.23} \approx 0.55$$

$$p(y = \text{cat} | x) = \frac{e^{0.8}}{e^{1.0} + e^{0.8}} \approx \frac{2.23}{2.72 + 2.23} \approx 0.45$$

Label : Dog

Quiz

Q1. Logistic Regression

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \theta_{\text{dog}} = \begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix} \quad \theta_{\text{cat}} = \begin{bmatrix} 0.6 \\ 0.1 \end{bmatrix} \quad z = \theta^T x$$

Q1-4. 실제 label이 Dog일 때 NLL Loss를 구하시오.

$$z_{\text{dog}} = \theta_{\text{dog}}^T x = 0.4 \times 1 + 0.3 \times 2 = 1.0$$

$$z_{\text{cat}} = \theta_{\text{cat}}^T x = 0.6 \times 1 + 0.1 \times 2 = 0.8$$

$$p(y = \text{dog}|x) = \frac{e^{1.0}}{e^{1.0} + e^{0.8}} \approx \frac{2.72}{2.72 + 2.23} \approx 0.55$$

$$p(y = \text{cat}|x) = \frac{e^{0.8}}{e^{1.0} + e^{0.8}} \approx \frac{2.23}{2.72 + 2.23} \approx 0.45$$

Label : Dog

$$\text{Loss} = -\log p_{\theta}(y = \text{dog}|x) = -\log 0.55 \approx 0.60$$

Quiz

Q2. Regularization

Model A	Training Loss : 0.8	$\theta = [3,4]$
Model B	Training Loss : 1.4	$\theta = [1,1]$

L2 regularization 사용 : $L_{total} = L_{train}(\theta) + \lambda \|\theta\|_2^2$, $\lambda = 0.1$

Prediction을 위해 Model A 선택

Q2-1. Model들의 regularization loss를 구하시오.

Q2-2. Model 선택이 바뀌어야하는지 판단하시오.

Quiz

Q2. Regularization

Model A	Training Loss : 0.8	$\theta = [3,4]$
Model B	Training Loss : 1.4	$\theta = [1,1]$

L2 regularization 사용 : $L_{total} = L_{train}(\theta) + \lambda \|\theta\|_2^2$, $\lambda = 0.1$

Prediction을 위해 Model A 선택

Q2-1. Model들의 regularization loss를 구하시오.

$$L_{total}(A) = 0.8 + 0.1 \times (3^2 + 4^2) = 0.8 + 2.5 = 3.3$$

$$L_{total}(B) = 1.4 + 0.1 \times (1^2 + 1^2) = 1.4 + 0.2 = 1.6$$

Quiz

Q2. Regularization

Model A	Training Loss : 0.8	$\theta = [3,4]$
Model B	Training Loss : 1.4	$\theta = [1,1]$

L2 regularization 사용 : $L_{total} = L_{train}(\theta) + \lambda \|\theta\|_2^2$, $\lambda = 0.1$

Prediction을 위해 Model A 선택

Q2-2. Model 선택이 바뀌어야하는지 판단하시오.

$$L_{total}(A) = 0.8 + 0.1 \times (3^2 + 4^2) = 0.8 + 2.5 = 3.3$$

$$L_{total}(B) = 1.4 + 0.1 \times (1^2 + 1^2) = 1.4 + 0.2 = 1.6$$

Model B 선택

